

Thermoelectric_prediction Package Documentation

Overview

thermoelectric_prediction is a Python package for analyzing and predicting thermoelectric properties. It includes three main modules:

DBSCANAnalyzer is an unsupervised analysis module for thermoelectric data that uses the DBSCAN algorithm to automatically group materials with similar properties and identify outliers. It enables the detection of patterns or natural subgroups within large multi-property datasets, while providing clear visualizations of clusters and atypical points. This module simplifies data exploration and helps highlight trends.

LinearRegressionAnalyzer is a module that analyzes and quantifies the linear relationship between two features. It fits a regression line, calculates the coefficient of determination (R^2), and provides clear visualizations to identify and illustrate linear trends between material properties.

ModelPredictor is a module designed to predict thermoelectric properties using machine learning models capable of capturing nonlinear relationships. It primarily leverages algorithms such as Random Forest, along with other advanced models, to learn complex patterns from data and generate accurate property predictions for materials.

Instruction

1. Before using any of these modules, make sure your data set is well prepared and clean.
2. Inside the [config.py](#) file, there are parameters defined for the example, if you are using different dataset, please change the parameters accordingly
3. Before using the third module, `other_model.py`, make sure you featurize your dataset. And you have the possibility to choose among several models: Random Forest; Gradient Boosting; SVR; KNN; Decision Tree.

Common Featurization Techniques

- **Magpie**: Used for composition-based feature generation.
- **SOAP (Smooth Overlap of Atomic Positions)**: A local descriptor for atomic environments.
- **Coulomb Matrix**: Represents nuclear charges and positions.

- **Crystal Graph Neural Networks (CGCNN):** Maps crystal structures as graphs, treating atoms as nodes and bonds as edges.

Installation

```
pip install -e .
```

Available Modules

1. DBSCANAnalyzer (**dbscan.py**)

Clustering of thermoelectric data

```
from thermoelectric_prediction import DBSCANAnalyzer, print_config
```

```
# Display configuration
print_config('DBSCAN')
```

```
# Initialize analyzer
analyzer = DBSCANAnalyzer()
```

```
# Load data
data = analyzer.load_data('data_dbscan.xlsx')
```

```
# Apply DBSCAN
labels = analyzer.fit()
```

```
# Analyze results
results = analyzer.analyze()
```

```
# Visualize
analyzer.plot_clusters(save_figure=True)
```

```
# Save individual clusters
saved_files = analyzer.save_clusters('data_dbscan.xlsx')
```

2. LinearRegressionAnalyzer (**linear_regression.py**)

Simple linear regression with one plot (predicted vs actual + $y=x$ line)

```
from thermoelectric_prediction import LinearRegressionAnalyzer

# Initialize (uses config.py automatically)
analyzer = LinearRegressionAnalyzer()

# Load data
analyzer.load_data('data_linear.xlsx')

# Perform regression
analyzer.fit() # Automatically uses parameters from config.py

# Single plot: predicted vs actual with  $y=x$  line
analyzer.plot()

# Simple summary
analyzer.summary()

# Make prediction
prediction = analyzer.predict(100)
print(f"Prediction for {analyzer.config['FEATURE']} = 100: {prediction:.2f}")
```

3. ModelPredictor (**other_model.py**)

```
from thermoelectric_prediction import ModelPredictor

# Initialiser pour prédire s_n avec Random Forest
predictor = ModelPredictor(target_column='s_n', model_type='random_forest')

# Charger et entraîner
X, y, df = predictor.load_data('other_data.xlsx')
results = predictor.train(X, y)

# Évaluer
predictor.evaluate()
predictor.plot_predictions()
```

```
predictor.plot_feature_importance()
```

```
# Sauvegarder le modèle  
predictor.save_model()
```

The practical examples for the three modules are provided through the Python scripts **dbscan.py**, **other_model.py**, and **linear_regression.py**. The parameter values used in these examples are defined in the **config** file; feel free to modify them to explore different outputs. You may also use your own dataset to run the analyses and predictions.

Typical Workflow

For all the calculations feel free to **Edit** `config.py` with your desired feature/target, and parameters. **Place** your data files in the same folder with different examples. **Run** the python script and **Check** results in `output/` folder

For Linear Regression:

Run: `python3 example_linear.py`

For DBSCAN:

Run: `python3 example_dbscan.py`

For Advanced Models:

Run: `python3 other_example.py`

